



INFORME DE GUÍA PRÁCTICA

I. PORTADA

Tema:	T2.7. arreglos ejercicios
Unidad de Organización Curricular:	BÁSICA
Nivel y Paralelo:	Primero - B
Alumno participante:	Chacha Chango Victor Manuel
Asignatura:	Algoritmos y Lógica de Programación
Docente:	Ing. José Caiza

II. INFORME DE GUÍA PRÁCTICA

2.1 Objetivos

General:

Desarrollar un sistema automatizado de escritorio mediante el uso de estructuras de control y arreglos para procesar, analizar y buscar información dentro de un conjunto de datos.

Específicos:

- Implementar bucles iterativos y arreglos (vectores) en lenguajes como C++ o Java para el registro y almacenamiento temporal de calificaciones.
- Diseñar algoritmos de cálculo estadístico aplicando acumuladores y contadores para determinar promedios y estados de aprobación.
- Aplicar el método de búsqueda secuencial para localizar registros específicos dentro de las estructuras de datos, validando coincidencias mediante banderas lógicas.

2.2 Modalidad

Presencial

2.3 Tiempo de duración

Presenciales: 6

No presenciales: 0

2.4 Instrucciones

- La tarea se debe realizar de forma grupal.
- Lea detenidamente cada uno de los ejercicios planteados en el aula virtual y desarrolle lo solicitado.
- Codifique en C++ o Java cada uno de los ejercicios planteados.
- Al finalizar, comprimir y subir al aula virtual el proyecto desarrollado, adicionalmente realizar capturas de cada una de las clases y métodos de los ejercicios. Las capturas
- se subirán en un archivo pdf en el formato de Guías APE.

2.5 Listado de equipos, materiales y recursos

Listado de equipos y materiales generales empleados en la guía práctica:



- Inteligencia artificial, TAC
- - Computador o laptop.
- - Plataforma educativa de la UTA.
- Internet para consultas.
- - Material de escritorio.

TAC (Tecnologías para el Aprendizaje y Conocimiento) empleados en la guía práctica:

- ☒ Plataformas educativas
- ☐ Simuladores y laboratorios virtuales
- ☐ Aplicaciones educativas
- ☒ Recursos audiovisuales
- ☐ Gamificación
- ☐ Inteligencia Artificial
- Otros (Especifique): _____

2.6 Actividades por desarrollar

EJERCICIO: SISTEMA DE GESTIÓN Y ANÁLISIS DE CALIFICACIONES

Enunciado: Una institución educativa necesita un sistema automatizado de escritorio para procesar las calificaciones del examen final de un grupo de estudiantes. Como ingeniero/a de software, se te ha encomendado construir un programa que recopile estos datos y genere un reporte estadístico detallado sobre el rendimiento del curso, además de permitir la búsqueda inmediata de información. Requerimientos: 1. Registrar Calificaciones, 2. Mostrar Reporte Estadístico (Promedio general, total aprobados ≥ 7 , total reprobados < 7), 3. Buscar Estudiante (Búsqueda Secuencial).

Análisis del problema:

Se debe construir un menú interactivo controlado por un ciclo do-while que permita navegar entre las opciones. Se utilizarán dos arreglos paralelos: uno de tipo cadena (String) para los nombres y uno de tipo real (double) para las notas. En la opción de reporte, se recorrerá el arreglo de notas con un bucle for, sumando valores y evaluando con if-else si la nota es ≥ 7 . Para la búsqueda, se comparará la cadena ingresada con los elementos del arreglo de nombres, usando una variable booleana para confirmar si se encontró o si se debe imprimir el mensaje de error.

Variables de entrada, salida y proceso:

- **Entrada:** opcion (Entero), cantidad (Entero), nombres (Arreglo Cadena), notas (Arreglo Real), buscar (Cadena).
- **Proceso:** Ciclo de menú, iteración for para llenar arreglos. $\text{suma} = \text{suma} + \text{notas}[i]$. $\text{promedio} = \text{suma} / \text{cantidad}$. Condicional $\text{notas}[i] \geq 7$. Búsqueda con validación de cadenas $\text{nombres}[i] == \text{buscar}$.
- **Salida:** Confirmaciones de registro, promedios y contadores numéricos en consola, y datos del estudiante encontrado o mensaje de error.

Algoritmo y Pseudocódigo (PSeInt):

Algoritmo SistemaCalificaciones

Definir opcion, i, cantidad, aprobados, reprobados Como Entero

Definir suma, promedio Como Real

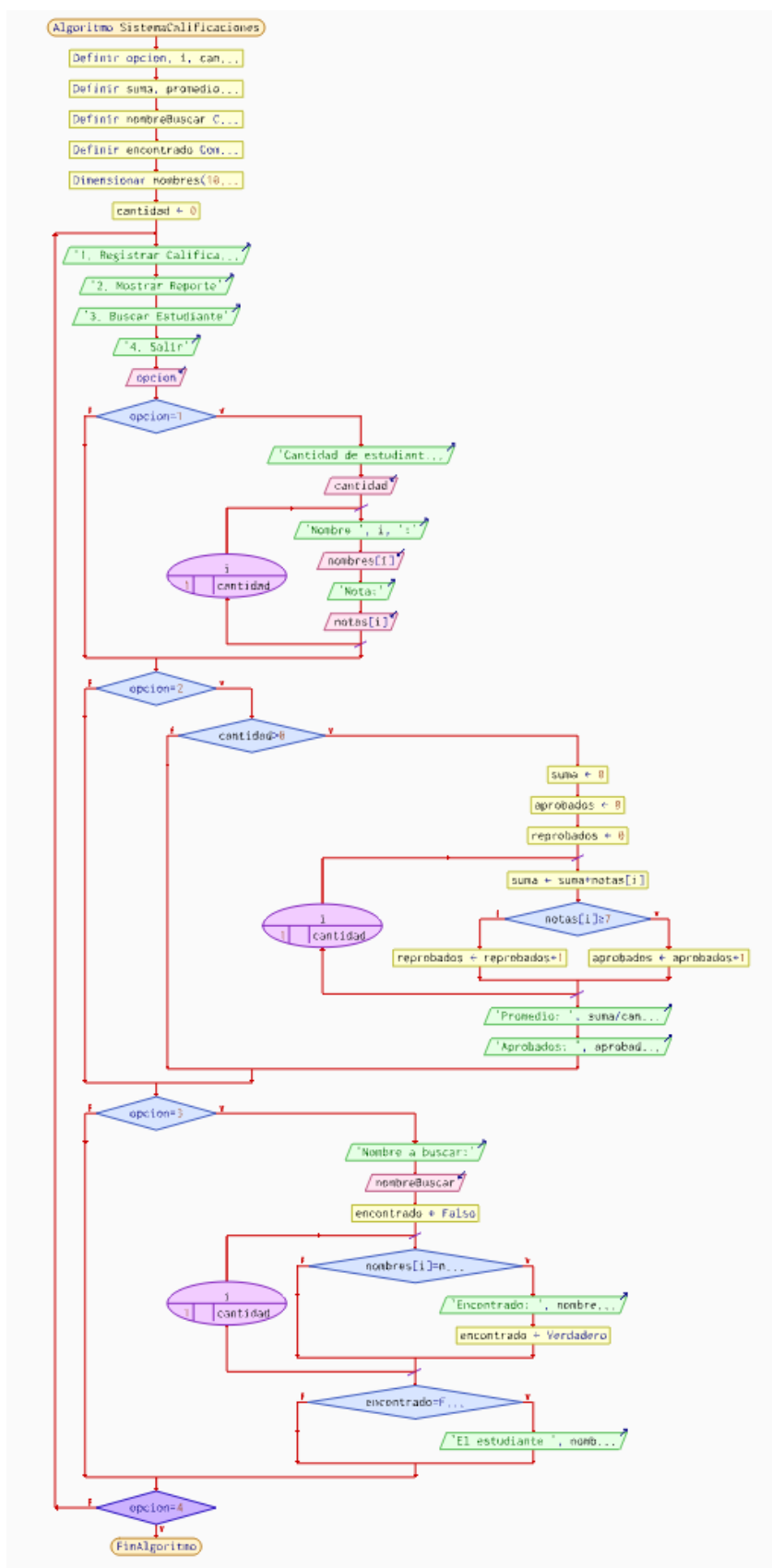


UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO – JULIO 2026



```
Definir nombreBuscar Como Cadena
Definir encontrado Como Logico
Dimension nombres[100], notas[100]
cantidad <- 0
Repetir
  Escribir "1. Registrar Calificaciones"
  Escribir "2. Mostrar Reporte"
  Escribir "3. Buscar Estudiante"
  Escribir "4. Salir"
  Leer opcion
  Si opcion = 1 Entonces
    Escribir "Cantidad de estudiantes:"
    Leer cantidad
    Para i <- 1 Hasta cantidad Hacer
      Escribir "Nombre ", i, ":"
      Leer nombres[i]
      Escribir "Nota:"
      Leer notas[i]
    FinPara
  FinSi
  Si opcion = 2 Entonces
    Si cantidad > 0 Entonces
      suma <- 0; aprobados <- 0; reprobados <- 0
      Para i <- 1 Hasta cantidad Hacer
        suma <- suma + notas[i]
        Si notas[i] >= 7 Entonces
          aprobados <- aprobados + 1
        Sino
          reprobados <- reprobados + 1
        FinSi
      FinPara
      Escribir "Promedio: ", suma / cantidad
      Escribir "Aprobados: ", aprobados, " | Reprobados: ",
reprobados
    FinSi
  FinSi
  Si opcion = 3 Entonces
    Escribir "Nombre a buscar:"
    Leer nombreBuscar
    encontrado <- Falso
    Para i <- 1 Hasta cantidad Hacer
      Si nombres[i] = nombreBuscar Entonces
        Escribir "Encontrado: ", nombres[i], " Nota: ", notas[i]
        encontrado <- Verdadero
      FinSi
    FinPara
    Si encontrado = Falso Entonces
      Escribir "El estudiante ", nombreBuscar, " no se encuentra
registrado."
    FinSi
  FinSi
  Hasta Que opcion = 4
FinAlgoritmo
```

Diagrama de Flujo:



*Imagen 1 (Diagrama de flujo en Pseint)
 Realizado por el grupo*



Código en C++:

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    string nombres[100];
    double notas[100];
    int cantidad = 0, opcion = 0;

    do {
        cout << "\n--- MENU ---\n1. Registrar\n2. Reporte\n3. Buscar\n4. Salir\nOpcion: ";
        cin >> opcion;

        if (opcion == 1) {
            cout << "Cantidad de alumnos: "; cin >> cantidad;
            for (int i = 0; i < cantidad; i++) {
                cout << "Nombre: "; cin >> nombres[i];
                cout << "Nota: "; cin >> notas[i];
            }
        } else if (opcion == 2 && cantidad > 0) {
            double suma = 0; int aprob = 0, reprob = 0;
            for (int i = 0; i < cantidad; i++) {
                suma += notas[i];
                if (notas[i] >= 7) aprob++; else reprob++;
            }
            cout << "Promedio: " << (suma/cantidad) << "\nAprobados: " << aprob
            << "\nReprobados: " << reprob << "\n";
        } else if (opcion == 3) {
            string buscar; cout << "Buscar nombre: "; cin >> buscar;
            bool enc = false;
            for (int i = 0; i < cantidad; i++) {
                if (nombres[i] == buscar) {
                    cout << "Nombre: " << nombres[i] << " | Nota: " << notas[i];
                    cout << " | Estado: " << (notas[i] >= 7 ? "Aprobado" :
                    "Reprobado") << "\n";
                    enc = true; break;
                }
            }
            if (!enc) cout << "El estudiante " << buscar << " no se encuentra registrado.\n";
        }
    } while (opcion != 4);
    return 0;
}
```



Código en Java:

```
import java.util.Scanner;

public class GestionCalificaciones {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String[] nombres = new String[100];
        double[] notas = new double[100];
        int cantidad = 0, opcion = 0;

        do {
            System.out.print("\n1. Registrar\n2. Reporte\n3. Buscar\n4. Salir\nOpcion: ");
            opcion = sc.nextInt();

            if (opcion == 1) {
                System.out.print("Cantidad: ");
                cantidad = sc.nextInt();
                for (int i = 0; i < cantidad; i++) {
                    System.out.print("Nombre: "); nombres[i] = sc.next();
                    System.out.print("Nota: "); notas[i] = sc.nextDouble();
                }
            } else if (opcion == 2 && cantidad > 0) {
                double suma = 0; int aprob = 0, reprob = 0;
                for (int i = 0; i < cantidad; i++) {
                    suma += notas[i];
                    if (notas[i] >= 7) aprob++; else reprob++;
                }
                System.out.println("Promedio: " + (suma/cantidad) +
                    "\nAprobados: " + aprob + "\nReprobados: " + reprob);
            } else if (opcion == 3) {
                System.out.print("Buscar: "); String buscar = sc.next();
                boolean enc = false;
                for (int i = 0; i < cantidad; i++) {
                    if (nombres[i].equalsIgnoreCase(buscar)) {
                        System.out.println(nombres[i] + " | Nota: " + notas[i]
                            + " | " + (notas[i] >= 7 ? "Aprobado" : "Reprobado"));
                        enc = true; break;
                    }
                }
                if (!enc) System.out.println("El estudiante " + buscar + " no se encuentra registrado.");
            }
        } while (opcion != 4);
    }
}
```



3 Casos de Prueba de Escritorio:

Caso de Prueba	Condición de Entrada	Proceso	Resultado en Salida
1. Registrar datos	Opción 1. Cant=2. Jose (9.5), Maria (6)	Almacena en índices 0 y 1 de los arreglos.	Confirmación implícita por menú.
2. Reporte Estadístico	Opción 2. Con datos previos.	Suma 15.5. Evaluaciones lógicas $9.5 \geq 7, 6 < 7$	Prom: 7.75, Aprobados: 1, Reprobados: 1
3. Búsqueda secuencial	Opción 3. Busca 'Pedro'	Recorre índices 0 y 1 sin coincidencias. Boleano = false.	"El estudiante Pedro no se encuentra registrado."

*Tabla 1 (Casos de prueba)
Realizado por el grupo*

2.7 Resultados obtenidos

Se logró implementar un sistema que recopila y almacena correctamente los nombres y las calificaciones de los estudiantes mediante el uso de dos arreglos paralelos (un arreglo de cadenas para los nombres y uno de reales para las notas).

El módulo de reportes procesó con éxito los datos almacenados, ejecutando un análisis estadístico dinámico mediante bucles for y condicionales if-else. Al evaluar las calificaciones con el criterio de aprobación, el sistema calculó de manera precisa el promedio general del curso (para los datos de prueba) y clasificó correctamente las métricas de rendimiento, arrojando como salida exacta: 1 estudiante aprobado y 1 estudiante reprobado.

e validó la funcionalidad de búsqueda inmediata de información mediante el método secuencial apoyado por banderas booleanas (true/false). El sistema demostró responder adecuadamente ante diferentes escenarios del usuario final: devolviendo los datos y el estado del alumno si se encuentra registrado, o mostrando un mensaje de error claro y controlado.

2.8 Habilidades blandas empleadas en la práctica

- ☐ Liderazgo
- ☒ Trabajo en equipo
- ☐ Comunicación asertiva
- ☐ La empatía
- ☒ Pensamiento crítico
- ☐ Flexibilidad
- ☐ La resolución de conflictos
- ☐ Adaptabilidad
- ☒ Responsabilidad

2.9 Conclusiones



La implementación de arreglos paralelos demostró ser una solución eficaz y estructurada para manejar registros múltiples de distintos tipos de datos (cadenas y numéricos) compartiendo un mismo índice lógico.

El uso de bucles de iteración como el `for` optimiza sustancialmente el proceso de análisis estadístico, permitiendo calcular promedios y contar estados lógicos recorriendo toda la estructura de datos con muy pocas líneas de código.

El método de búsqueda secuencial, apoyado por banderas lógicas (booleanas), garantiza que el sistema pueda validar la existencia o ausencia de un registro de manera precisa, mejorando la experiencia del usuario final al proveer mensajes de error claros cuando no hay coincidencias.

2.10 Recomendaciones

Asegurarse de inicializar correctamente los contadores y acumuladores (en cero) antes de iniciar el ciclo `for` en el módulo de reportes, para evitar resultados erróneos o acarreo de basura en la memoria.

Implementar validaciones adicionales en la opción de registro para asegurar que las calificaciones ingresadas por el usuario se mantengan obligatoriamente dentro del rango válido (por ejemplo, entre 0 y 10).

En el proceso de búsqueda (especialmente en Java), utilizar métodos de comparación de cadenas como `.equalsIgnoreCase()` en lugar del operador `==`, para evitar fallos de coincidencia causados por difer

2.11 Referencias bibliográficas

- Deitel, P., & Deitel, H. (2018). *Java: Cómo programar* (11.^a ed.). Pearson Educación.
- Joyanes Aguilar, L. (2020). *Fundamentos de programación: Algoritmos, estructuras de datos y objetos* (6.^a ed.). McGraw-Hill.
- Stroustrup, B. (2018). *Programación: Principios y práctica usando C++* (2.^a ed.). Pearson Educación.

2.12 Anexos

Link de Github

<https://github.com/VictorChacha1/-T2.7.-arreglos-ejercicios.git>

Evidencias del código



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO – JULIO 2026



```
1. Registrar
2. Reporte
3. Buscar
4. Salir
Opcion: 11
```

```
1. Registrar
2. Reporte
3. Buscar
4. Salir
Opcion: 15
```

*Imagen 2 (Bucle si hay errores en el código)
Realizado por el grupo*

```
1. Registrar
2. Reporte
3. Buscar
4. Salir
Opcion: 1
Cantidad: 3
Nombre: Jose
Nota: 10
Nombre: Luis
Nota: 7
Nombre: Maria
Nota: 9
```

*Imagen 2 (Registro de estudiantes)
Realizado por el grupo*

```
1. Registrar
2. Reporte
3. Buscar
4. Salir
Opcion: 2
Promedio: 8.666666666666666
Aprobados: 3
Reprobados: 0
```

*Imagen 2 (Generación del reporte de los estudiantes ingresados)
Realizado por el grupo*



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO – JULIO 2026



```
1. Registrar
2. Reporte
3. Buscar
4. Salir
Opcion: 3
Buscar: Luis
Luis | Nota: 7.0 | Aprobado
```

Imagen 2 (Buscar al estudiante ingresado por el nombre)
Realizado por el grupo



INFOGRAFÍA

SISTEMA AUTOMATIZADO DE GESTIÓN DE CALIFICACIONES

PROPÓSITO: Optimizar y automatizar el proceso de registro, análisis y búsqueda de calificaciones académicas, fomentando la eficiencia educativa y el acceso rápido a la información en tiempo real.



FUNCIONAMIENTO

1

REGISTRO

Ingreso de datos de estudiantes y calificaciones a través de una interfaz intuitiva.



2

ANÁLISIS ESTADÍSTICO

Procesamiento automático de calificaciones para generar informes, promedios y tendencias de rendimiento.



3

BÚSQUEDA

Acceso rápido y seguro a perfiles de estudiantes y sus historiales académicos mediante filtros avanzados.

